

SE446: Big Data Engineering

Week 9A: Spark MLlib — Machine Learning at Scale

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

- 1 From Traditional ML to Distributed ML
- 2 The ML Pipeline API
- 3 Feature Engineering
- 4 Model Training
- 5 Model Evaluation
- 6 Model Comparison & Hyperparameter Tuning
- 7 Summary

Learning Objectives

By the end of this session you will be able to:

- 1 Explain **why** traditional ML breaks at scale and how distributed ML solves it
- 2 Describe the ML Pipeline API: `Transformer`, `Estimator`, `Pipeline`
- 3 Prepare features with `StringIndexer` and `VectorAssembler`
- 4 Train classification models (Logistic Regression, Random Forest, GBT)
- 5 Evaluate with AUC-ROC, accuracy, F1, precision, recall
- 6 Tune hyperparameters with `CrossValidator`
- 7 Interpret feature importances to understand model decisions

Today's Task

Build an ML pipeline to predict whether a crime results in **arrest** using the Chicago Crimes dataset — the same dataset you've been analyzing all semester.

Outline

- 1 From Traditional ML to Distributed ML
- 2 The ML Pipeline API
- 3 Feature Engineering
- 4 Model Training
- 5 Model Evaluation
- 6 Model Comparison & Hyperparameter Tuning
- 7 Summary

You Already Know Machine Learning

*This is **not** your first ML course. Let's build on what you know.*



What You Already Know

- Train/test split
- Feature engineering
- Logistic Regression, Random Forest
- Accuracy, Precision, Recall, F1
- Cross-validation, GridSearch
- scikit-learn API



What's New Today

- **Why** scikit-learn breaks at scale
- **How** distributed training works
- The Spark MLlib Pipeline API
- Same algorithms, different execution
- Training on terabytes, not gigabytes
- When to use which tool



Today's Guiding Question

Same math, same intuition — but what changes when your data doesn't fit on one machine?

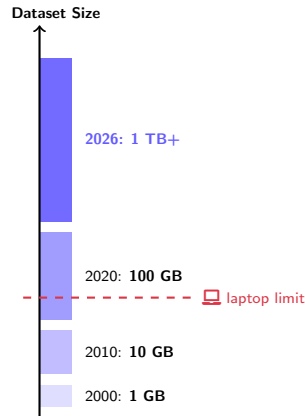
ML Is Everywhere — But Data Is Getting Bigger

Real problems that use ML on large data:

-  **Crime prediction:** Can we predict whether a crime will lead to arrest? (*Today's task*)
-  **Healthcare:** Detect disease from millions of patient records
-  **Finance:** Fraud detection on billions of transactions per day
-  **Web:** Recommendation systems for 100M+ users

The Challenge

These datasets do **not fit on a laptop**.
A single machine crashes, stalls, or takes days.



What Breaks When You Try scikit-learn on Big Data

Your Laptop + scikit-learn

What actually happens

```
pd.read_csv("crimes.csv")  
150 GB file
```

MemoryError: Unable to
allocate 150 GB

```
model.fit(X_train)  
40M rows
```

Running...8 hours
(single CPU thread)

```
GridSearchCV(cv=10, ...)  
50 param combos
```

Estimated time: 4 days
on 1 machine

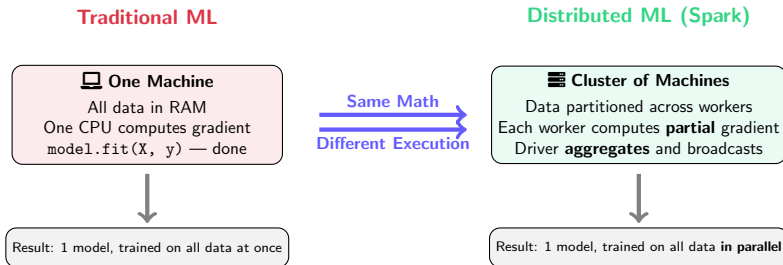
The Root Cause — Three Walls

Memory wall: scikit-learn loads **all data into one machine's RAM**.

Compute wall: Trains on **one CPU** (or a few cores at best).

Scale wall: There is **no way to add more machines** — you're stuck.

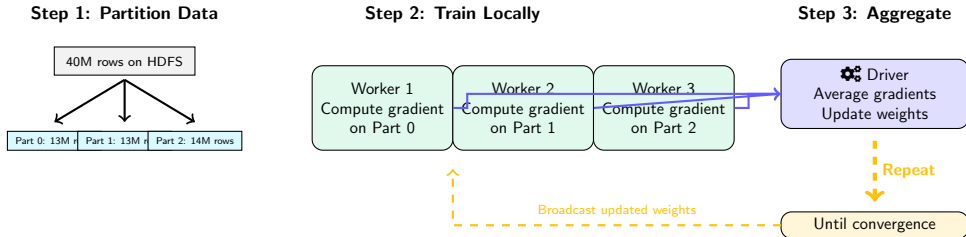
Traditional ML vs Distributed ML — What Actually Changes



The Fundamental Shift

The **algorithm** (e.g., gradient descent) is the same. The **execution** changes: data is split, gradients are computed locally, then **aggregated** on the driver. You don't learn new math — you learn a new **execution model**.

How Spark Actually Trains a Model



Why This Is Faster

3 workers each process 13M rows = **3× faster** than 1 machine on 40M rows. Add more workers → near-linear speedup. **Same math, parallel execution.**

Spark MLlib — Same Idea, Scales Across the Cluster

scikit-learn (1 machine)

```
import pandas as pd
from sklearn.ensemble import (
    RandomForestClassifier)
from sklearn.model_selection import (
    train_test_split)

df = pd.read_csv("crimes.csv")
X = df[["District", "Hour"]]
y = df["Arrest"]
X_tr, X_te, y_tr, y_te = (
    train_test_split(X, y, test_size=0.2))

model = RandomForestClassifier()
model.fit(X_tr, y_tr)
```

✗ Crashes on large data

Spark MLlib (cluster)

```
from pyspark.ml.classification import (
    RandomForestClassifier)
from pyspark.ml import Pipeline

df = spark.read.csv("hdfs:///...")
train_df, test_df = (
    df.randomSplit([0.8, 0.2]))

rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="label")
pipeline = Pipeline(
    stages=[indexer, assembler, rf])

model = pipeline.fit(train_df)
```

✓ Runs on terabytes, any cluster

The Key Difference

scikit-learn: data is a numpy array in RAM → one machine does everything.

MLlib: data is a distributed DataFrame on HDFS → cluster works in parallel.

Your scikit-learn Knowledge → MLlib Translation Guide

Concept	scikit-learn	Spark MLlib
Load data	<code>pd.read_csv()</code>	<code>spark.read.csv()</code>
Data structure	DataFrame (pandas)	DataFrame (Spark)
Feature vector	numpy array / column list	VectorAssembler
Encode categories	LabelEncoder	StringIndexer
One-hot encoding	OneHotEncoder	OneHotEncoder
Train/test split	<code>train_test_split()</code>	<code>df.randomSplit()</code>
Chain steps	<code>sklearn.pipeline</code>	<code>pyspark.ml.Pipeline</code>
Train	<code>model.fit(X, y)</code>	<code>pipeline.fit(df)</code>
Predict	<code>model.predict(X)</code>	<code>model.transform(df)</code>
Grid search	GridSearchCV	CrossValidator
Save model	<code>joblib.dump()</code>	<code>model.save("hdfs://...")</code>

Notice the Pattern

The **concepts** are **identical**. The API names are slightly different, and the data lives on a **cluster instead of RAM**.

When to Use scikit-learn vs Spark MLlib

Use scikit-learn

- ✓ Data fits in RAM (<5–10 GB)
- ✓ Rapid prototyping / exploration
- ✓ Need exotic algorithms (XGBoost, SVM kernels, DBSCAN, etc.)
- ✓ Small team, quick iteration

Use Spark MLlib

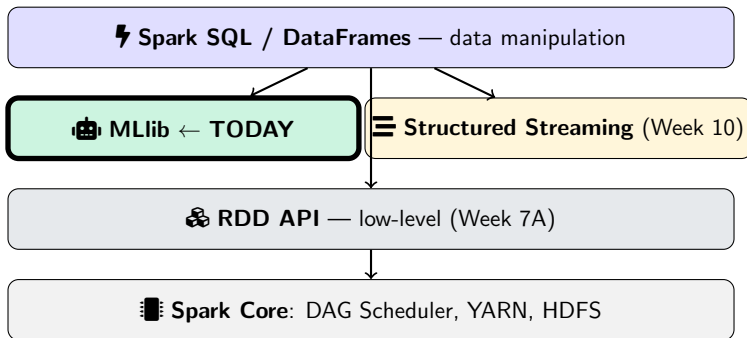
- ✓ Data is too large for one machine
- ✓ Data already lives on HDFS/S3
- ✓ Need to train on 10 GB+ regularly
- ✓ Production pipeline at scale
- ✓ Need parallel hyperparameter tuning

Decision: data size + infrastructure

Common Mistake

Distributed $\neq$ always better. Spark has **overhead** (serialization, shuffle, coordination). On small data, scikit-learn is **faster**. Use the right tool for the right scale.

MLlib in the Spark Ecosystem



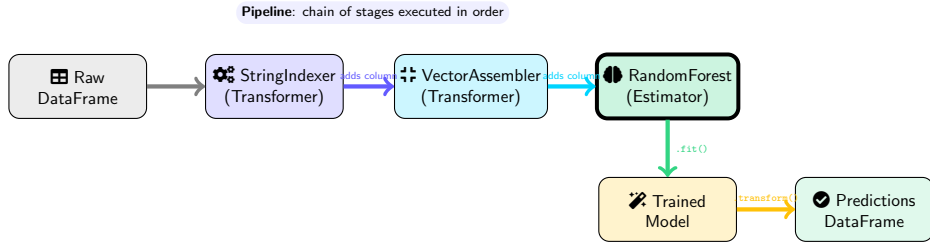
API Warning

`pyspark.mllib` (RDD-based) — **old, maintenance mode, do NOT use.**
`pyspark.ml` (DataFrame-based) — **current, recommended.** We use this.

Outline

- 1 From Traditional ML to Distributed ML
- 2 The ML Pipeline API
- 3 Feature Engineering
- 4 Model Training
- 5 Model Evaluation
- 6 Model Comparison & Hyperparameter Tuning
- 7 Summary

Pipeline Concept — Assembly Line



Transformer

Takes a DF, returns a DF

Method: `.transform(df)`

Example: VectorAssembler

Estimator

Takes a DF, **learns** params

Method: `.fit(df) → Model`

Example: RandomForest

Transformer vs Estimator — Detailed Comparison

	Transformer	Estimator
Method	<code>.transform(df) → df</code>	<code>.fit(df) → Model (Transformer)</code>
Parameters	Fixed (no learning)	Learned from data
Examples	VectorAssembler, OneHotEncoder (fitted), StringIndexer (fitted)	LogisticRegression, RandomForest, StringIndexer (unfitted)
sklearn analogy	<code>.transform()</code> after <code>.fit()</code>	<code>.fit()</code> itself
Analogy	A ruler (measures directly)	A student (learns, then measures)

Important Nuance

StringIndexer starts as an **Estimator** (learns category → index mapping via `.fit()`), then produces a fitted **Transformer** (applies the mapping via `.transform()`). Same as `sklearn.preprocessing.LabelEncoder` — `.fit()` then `.transform()`.

Why Pipelines Matter **More** in Distributed ML

✗ Without Pipeline (danger zone)

- Fit StringIndexer on full data → **data leakage**
- Manually apply each step → **inconsistent** train vs test
- Can't serialize to HDFS → **can't deploy** to cluster
- Each worker needs transforms → **manual coordination**

✓ With Pipeline

- `pipeline.fit(train_df)` → fits **only on train**
- `model.transform(test_df)` → **same steps** automatically
- `model.save("hdfs://...")` → **any worker** can load
- Pipeline ships transforms to every worker **automatically**

```
pipeline = Pipeline(stages=[
    string_indexer,      # Stage 1: encode categories
    vector_assembler,    # Stage 2: assemble features
    classifier            # Stage 3: train model
])
model = pipeline.fit(train_df)      # fit everything
predictions = model.transform(test_df) # apply everything
```

🔑 In scikit-learn you *can* skip pipelines. In Spark, you **shouldn't**.

Distributed data across nodes means transforms must travel **with** the model.

Without a Pipeline — The Code You'd Have to Write

What does it actually look like when you skip the Pipeline and do everything manually?

```
# WRONG: fit on full data BEFORE splitting => data leakage!
indexer = StringIndexer(inputCol="Primary Type",
                        outputCol="crime_index")
indexer_model = indexer.fit(df)           # sees test data too!
df = indexer_model.transform(df)

assembler = VectorAssembler(
    inputCols=["District", "crime_index", "Hour"],
    outputCol="features")
df = assembler.transform(df)

train_df, test_df = df.randomSplit([0.8, 0.2])

rf = RandomForestClassifier(featuresCol="features",
                           labelCol="label")
model = rf.fit(train_df)
predictions = model.transform(test_df)

# Now you need to deploy...
# Save the indexer? the assembler? the model? separately?
indexer_model.save("hdfs:///models/indexer") # manual
model.save("hdfs:///models/rf_model")        # manual
# Every worker must load BOTH in the right order!
```

⚠ Three Problems Here

1. StringIndexer fitted on full data → **data leakage** (test info leaks into train). 2. Must save/load each stage separately → **error-prone deployment**. 3. Workers must manually coordinate transform order → **fragile at scale**.

Outline

- 1 From Traditional ML to Distributed ML
- 2 The ML Pipeline API
- 3 Feature Engineering**
- 4 Model Training
- 5 Model Evaluation
- 6 Model Comparison & Hyperparameter Tuning
- 7 Summary

Our Dataset: Chicago Crimes

Column	Type	Role in ML
District	int	✓ Feature (numeric)
Primary Type	string	✓ Feature (categorical → encode)
Hour (extracted)	int	✓ Feature (numeric)
Domestic	boolean	✓ Feature (categorical → encode)
Arrest	boolean	🚩 Label (what we predict)

© **Goal:** Given District, Crime Type, Hour, Domestic → predict **Arrest** (True/False)

Why These Features? Think Before You Code

*In your ML course you learned: feature selection should be driven by **domain knowledge**, not guessing.*

Feature	Domain Intuition	Expected Effect
Primary Type	NARCOTICS → almost always arrest. THEFT → rarely catch in act.	Strong predictor — crime type determines police procedure
Domestic	Many jurisdictions have mandatory arrest policies for domestic crimes	Strong predictor — policy-driven
District	Some districts have more resources, higher patrol density	Moderate predictor — varies by staffing
Hour	Crimes at 2 AM: fewer witnesses, harder to catch. vs 2 PM: more police visible	Moderate predictor — time affects opportunity

⚠ What We're NOT Using (and Why)

Block (address) → too many unique values, would overfit. **Date** → raw string, but we extract **Hour** as a useful signal. **Year** → could capture trend but risks data leakage on temporal data.

Step 1: Extract Features from Raw Data

In scikit-learn: `df["Hour"] = pd.to_datetime(df["Date"]).dt.hour`. In Spark:

```
from pyspark.sql.functions import hour, to_timestamp, col

# Load dataset from HDFS (distributed!)
df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)

# Extract hour of day from Date string
df = df.withColumn("Hour",
                  hour(to_timestamp(col("Date"),
                                   "MM/dd/yyyy hh:mm:ss a"))))

# Select relevant columns, drop nulls
df = df.select("District", "Primary Type",
               "Hour", "Domestic", "Arrest").dropna()

# Cast label to integer (True=1, False=0)
df = df.withColumn("label",
                  col("Arrest").cast("integer"))
```

Tip

Always check for nulls with `df.filter(col("Hour").isNull()).count()` before training. Nulls silently break ML models.

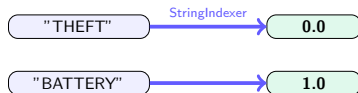
Step 2: StringIndexer — Encoding Categories

In scikit-learn: `LabelEncoder().fit_transform(df["Primary Type"])`. In Spark:

```
from pyspark.ml.feature import StringIndexer

# "THEFT" -> 0.0, "BATTERY" -> 1.0, ...
crime_indexer = StringIndexer(
    inputCol="Primary Type",
    outputCol="crime_index",
    handleInvalid="skip"    # drop rows with unseen categories
)

# "false" -> 0.0, "true" -> 1.0
domestic_indexer = StringIndexer(
    inputCol="Domestic",
    outputCol="domestic_index",
    handleInvalid="skip"
)
```



Assigns indices by
frequency (most common = 0)

StringIndexer vs OneHotEncoder — When to Use Which

StringIndexer Only

"THEFT" → 0.0
"BATTERY" → 1.0
"ROBBERY" → 2.0

StringIndexer + OneHotEncoder

"THEFT" → (1, 0, 0)
"BATTERY" → (0, 1, 0)
"ROBBERY" → (0, 0, 1)

	StringIndexer only	+ OneHotEncoder
Use with	✓ Tree models (RF, GBT)	✓ Linear models (LR)
Why?	Trees split on values, no order assumed	LR treats 2.0 as "more" than 1.0 → wrong
Feature count	1 column per category	$N - 1$ columns (sparse)

 Rule of Thumb (Same as scikit-learn!)

Tree-based models → StringIndexer is enough. **Linear models** → add OneHotEncoder to avoid false ordinal relationships.

Step 3: VectorAssembler — Combine into Feature Vector

In scikit-learn: $X = df[["District", "crime_index", "Hour", "domestic_index"]]$. In Spark:

```
from pyspark.ml.feature import VectorAssembler

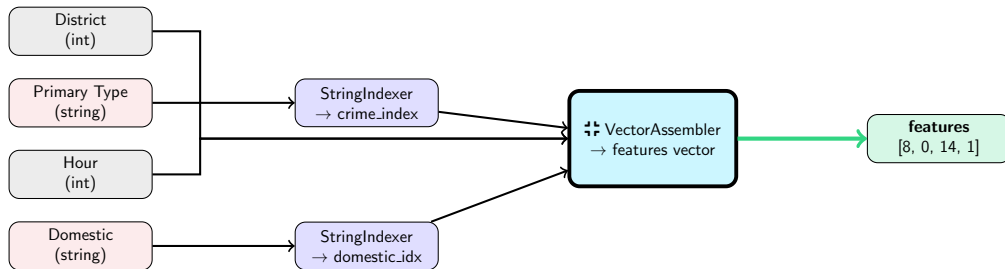
assembler = VectorAssembler(
    inputCols=["District", "crime_index",
               "Hour", "domestic_index"],
    outputCol="features"
)
```

District	crime_index	Hour	domestic.idx		features
8	0.0	14	1.0	→	[8, 0, 14, 1]
11	1.0	22	0.0		[11, 1, 22, 0]

Why a Single Vector?

All Spark ML algorithms require input as a **single vector column** called features. This is different from scikit-learn where you pass a matrix. VectorAssembler creates a DenseVector per row.

Feature Engineering Summary

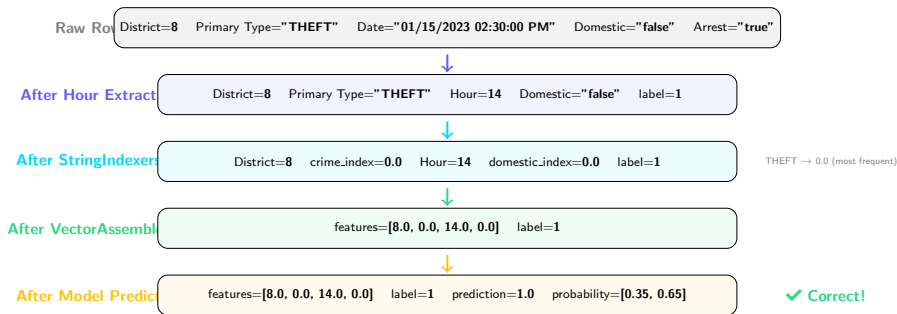


Pipeline Order Matters

StringIndexers must run **before** VectorAssembler, because the assembler needs the numeric index columns to exist. Same logic as scikit-learn's ColumnTransformer ordering.

One Row Through the Pipeline — Step by Step

Let's trace a single Chicago crime record through every stage:



Every Row Goes Through This

The pipeline applies these 4 stages to **every row in parallel across the cluster**. On 40M rows with 4 workers, each worker processes ~10M rows simultaneously.

Outline

- 1 From Traditional ML to Distributed ML
- 2 The ML Pipeline API
- 3 Feature Engineering
- 4 Model Training**
- 5 Model Evaluation
- 6 Model Comparison & Hyperparameter Tuning
- 7 Summary

Train/Test Split

In scikit-learn: `train_test_split(X, y, test_size=0.2)`. In Spark:

```
# 80% train, 20% test
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)

print(f"Training: {train_df.count()} rows")
print(f"Testing: {test_df.count()} rows")

# IMPORTANT: Check class balance!
train_df.groupBy("label").count().show()
```

Class Imbalance Warning

If 85% of crimes have Arrest=False, a model can just predict False and get 85% accuracy! → Always check F1 and AUC, not just accuracy.

seed=42

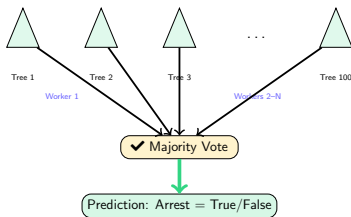
Fixed seed ensures **reproducible** splits — same rows in train/test every time you run. Same concept as `random_state=42` in sklearn.

Random Forest Classifier

*Same algorithm you know from your ML course — but Spark trains trees **in parallel across workers**.*

```
from pyspark.ml.classification import RandomForestClassifier

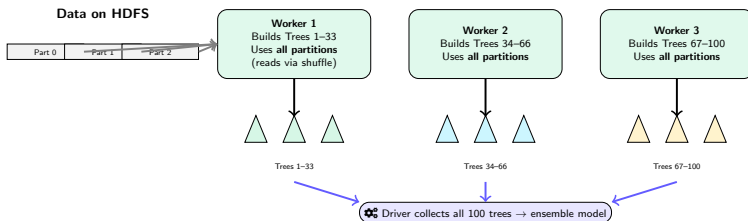
rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="label",
    numTrees=100,      # 100 decision trees
    maxDepth=5,        # max depth per tree
    seed=42
)
```



Spark distributes tree training across workers — each worker builds subset of trees in parallel

How Spark Parallelizes Random Forest — What Runs Where?

Students often ask: does each worker build a full tree? Or each partition?



Common Misconception

- ✗ “Each partition builds one tree”
- ✓ Each **worker** builds a **subset of trees**, but each tree sees **all the data** (via bootstrap sampling across partitions).

Train/Test on a Cluster

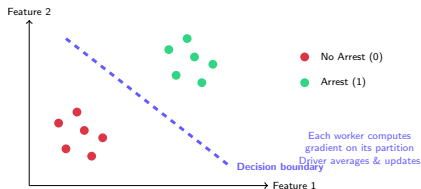
`randomSplit([0.8,0.2])` assigns each **row** (not partition!) to train or test using a hash. Both splits remain distributed across all nodes.

Logistic Regression

Same sigmoid function — but gradient descent runs **across workers**, each computing partial gradients.

```
from pyspark.ml.classification import LogisticRegression

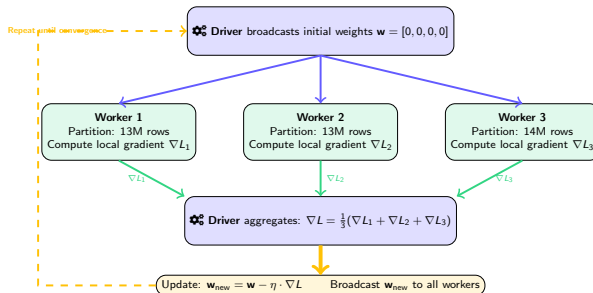
lr = LogisticRegression(
    featuresCol="features",
    labelCol="label",
    maxIter=100,      # max gradient descent steps
    regParam=0.01     # L2 regularization strength
)
```



When to Use

Fast baseline model. Works best for linearly separable data. Always start here, then try Random Forest or GBT.

Distributed Gradient Descent — How Spark Trains Logistic Regression



Key Insight

Each worker computes a gradient on its **local partition only**. The driver averages all gradients and updates the global weights. Workers never see each other's data — only gradient vectors are shipped (small!).

Distributed Gradient Descent — Numerical Example

Let's trace one iteration with real numbers. 2 features, 2 workers, learning rate $\eta = 0.1$.

Step	Worker 1 (Part. 0)	Worker 2 (Part. 1)
Data (2 rows each)	$x_1 = [1, 2], y_1 = 1$ $x_2 = [2, 1], y_2 = 0$	$x_3 = [3, 1], y_3 = 1$ $x_4 = [1, 3], y_4 = 0$
Receive weights	$\mathbf{w} = [0.5, 0.3]$	
Compute $\hat{y} = \sigma(\mathbf{w} \cdot \mathbf{x})$	$\hat{y}_1 = \sigma(1.1) = 0.75$ $\hat{y}_2 = \sigma(1.3) = 0.79$	$\hat{y}_3 = \sigma(1.8) = 0.86$ $\hat{y}_4 = \sigma(1.4) = 0.80$
Local gradient $\nabla L_k = \frac{1}{n_k} \sum (\hat{y} - y)\mathbf{x}$	$\nabla L_1 = [-0.045, 0.145]$	$\nabla L_2 = [0.170, 0.330]$

Driver Aggregates and Updates

$$\nabla L = \frac{1}{2}(\nabla L_1 + \nabla L_2) = \frac{1}{2}([-0.045, 0.145] + [0.170, 0.330]) = [\mathbf{0.0625}, \mathbf{0.2375}]$$

$$\mathbf{w}_{\text{new}} = [0.5, 0.3] - 0.1 \times [0.0625, 0.2375] = [\mathbf{0.494}, \mathbf{0.276}]$$

Broadcast $\mathbf{w}_{\text{new}}$ to all workers $\rightarrow$ repeat for next iteration.

🔑 What Travels Over the Network?

Only the weight vector $\mathbf{w}$ (4 floats) and gradient vectors — **not** the data. Data stays local. This is why distributed gradient descent scales.

Gradient-Boosted Trees (GBT) — Often the Best Performer

You may know XGBoost from your ML course. *GBTClassifier* is Spark's distributed equivalent.

```
from pyspark.ml.classification import GBTClassifier

gbt = GBTClassifier(
    featuresCol="features",
    labelCol="label",
    maxIter=50,           # number of boosting rounds
    maxDepth=5,           # depth per tree
    stepSize=0.1,         # learning rate
    seed=42
)
```



RF: Trees are independent (bagging) → easy to parallelize
GBT: Trees are sequential (boosting) → each corrects previous errors
GBT often wins on accuracy, RF is faster to train

GBT Limitation in Spark

Spark's *GBTClassifier* currently only supports **binary classification**. For multiclass, use Random Forest or Logistic Regression.

Complete Pipeline — Putting It Together

```
from pyspark.ml import Pipeline

pipeline = Pipeline(stages=[
    crime_indexer,      # "Primary Type" -> crime_index
    domestic_indexer,   # "Domestic" -> domestic_index
    assembler,          # columns -> features vector
    rf                  # RandomForestClassifier
])

# Cache training data (iterative training benefits!)
train_df.cache()

# Train the entire pipeline
model = pipeline.fit(train_df)

# Predict on test data
predictions = model.transform(test_df)
predictions.select("features", "label",
                   "prediction", "probability").show(5)
```

Key Principle

`pipeline.fit(train_df)` fits **all stages** in order. `model.transform(test_df)` applies **all stages** in order. Compare with sklearn: `pipe.fit(X_train)` then `pipe.predict(X_test)` — same idea!

Outline

- 1 From Traditional ML to Distributed ML
- 2 The ML Pipeline API
- 3 Feature Engineering
- 4 Model Training
- 5 Model Evaluation**
- 6 Model Comparison & Hyperparameter Tuning
- 7 Summary

What Predictions Look Like

features	label	prediction	probability	
[8, 0, 14, 1]	0	0.0	[0.82, 0.18]	✓
[11, 1, 22, 0]	1	1.0	[0.35, 0.65]	✓
[3, 2, 3, 1]	1	0.0	[0.55, 0.45]	✗

prediction

Model's binary answer: 0.0 or 1.0
(*sklearn: model.predict()*)

probability

Confidence: [P(class=0), P(class=1)]
(*sklearn: model.predict_proba()*)

Evaluation Metrics — You Know These!

Same metrics from your ML course. The formulas don't change in distributed ML.

Metric	Formula	Meaning for Our Task
Accuracy	$\frac{TP+TN}{\text{Total}}$	% of correct arrest predictions
Precision	$\frac{TP}{TP+FP}$	Of predicted arrests, how many were real?
Recall	$\frac{TP}{TP+FN}$	Of real arrests, how many did we catch?
F1 Score	$\frac{2 \cdot P \cdot R}{P+R}$	Balance between P and R
AUC-ROC	Area under ROC curve	Overall ranking quality (0.5 = random)

Why Not Just Accuracy? (Refresher)

Chicago crimes: ~85% have No Arrest. A model predicting “No Arrest” for everything gets **85% accuracy** but catches **zero criminals**. Always check **F1** and **AUC**.

Distributed Metric Computation

Spark computes TP/TN/FP/FN counts **per partition**, then aggregates across workers. Same formulas, parallel counting.

Confusion Matrix — Visual Guide

		Predicted	
		No Arrest (0)	Arrest (1)
Actual	No Arrest (0)	TN 6800	FP 350
	Arrest (1)	FN 420	TP 1430

TN: Correctly said No Arrest ✓

FP: False alarm — wasted resources ✗

FN: Missed arrest — criminal walks free ✗

TP: Correctly predicted arrest ✓

Reading the Matrix

$$\text{Precision} = \frac{1430}{1430+350} = 0.80 \quad \text{Recall} = \frac{1430}{1430+420} = 0.77 \quad \text{F1} = \frac{2 \times 0.80 \times 0.77}{0.80+0.77} = 0.78$$

Computing Metrics in Spark

In *scikit-learn*: `accuracy_score()`, `f1_score()`, etc. In *Spark*:

```
from pyspark.ml.evaluation import \
    BinaryClassificationEvaluator, \
    MulticlassClassificationEvaluator

# AUC-ROC
binary_eval = BinaryClassificationEvaluator(labelCol="label")
auc = binary_eval.evaluate(predictions)
print(f"AUC-ROC: {auc:.4f}")

# Accuracy, F1, Precision, Recall
mc_eval = MulticlassClassificationEvaluator(
    labelCol="label", predictionCol="prediction")

accuracy = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "accuracy"})
f1 = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "f1"})
precision = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "weightedPrecision"})
recall = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "weightedRecall"})
print(f"Accuracy: {accuracy:.4f}  F1: {f1:.4f}")
print(f"Precision: {precision:.4f}  Recall: {recall:.4f}")

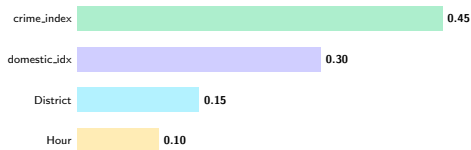
# Confusion matrix
predictions.groupBy("label", "prediction") \
    .count().orderBy("label", "prediction").show()
```

Feature Importances — Which Features Drive Predictions?

Same concept as `sklearn's model.feature_importances_`:

```
# Extract the fitted Random Forest model
rf_model = model.stages[-1] # last pipeline stage
importances = rf_model.featureImportances

feature_names = ["District", "crime_index",
                 "Hour", "domestic_index"]
for name, imp in zip(feature_names, importances):
    print(f" {name}: {imp:.4f}")
```



Interpretation — Matches Our Domain Reasoning!

Crime type is the strongest predictor (NARCOTICS → almost always arrest). **Domestic flag** second (mandatory arrest policies). This confirms our domain intuition from the feature selection slide.

Outline

- 1 From Traditional ML to Distributed ML
- 2 The ML Pipeline API
- 3 Feature Engineering
- 4 Model Training
- 5 Model Evaluation
- 6 Model Comparison & Hyperparameter Tuning**
- 7 Summary

Logistic Regression vs Random Forest vs GBT

Aspect	Logistic Reg.	Random Forest	GBT
Training speed	✓ Fastest	Moderate	Slowest (sequential)
Accuracy (typical)	Baseline	Good	✓ Often best
Non-linear	✗ No	✓ Yes	✓ Yes
Interpretability	Coefficients	Feature importances	Feature importances
Overfitting risk	Low	Moderate	⚠ High
Parallelism	✓ Gradient parallel	✓ Trees parallel	⚠ Limited
sklearn equivalent	LogisticRegression	RandomForest	XGBoost / GBM

Recommended Strategy (Same as Traditional ML!)

Step 1: Logistic Regression (fast baseline). **Step 2:** Random Forest (handles non-linearity). **Step 3:** GBT (squeeze out extra accuracy).

If $\text{GBT} \gg \text{LR}$, the relationship is non-linear. If $\text{LR} \approx \text{RF}$, keep LR (simpler).

Hyperparameter Tuning — CrossValidator

In *scikit-learn*: *GridSearchCV*. In *Spark*: *CrossValidator* + *ParamGridBuilder*.

The big difference? *Spark* runs folds in parallel across the cluster.

```
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

# Define parameter grid (like sklearn's param_grid)
paramGrid = ParamGridBuilder() \
    .addGrid(rf.numTrees, [50, 100, 200]) \
    .addGrid(rf.maxDepth, [3, 5, 10]) \
    .build() # 3 x 3 = 9 combinations

# CrossValidator (like sklearn's GridSearchCV)
cv = CrossValidator(
    estimator=pipeline,          # our full pipeline
    estimatorParamMaps=paramGrid,
    evaluator=BinaryClassificationEvaluator(labelCol="label"),
    numFolds=3,                  # 3-fold cross-validation
    parallelism=4                # run 4 folds in parallel!
)

# Fit --- Spark tries all 9 combos x 3 folds = 27 models
cvModel = cv.fit(train_df)

# Best model is automatically selected
best_predictions = cvModel.transform(test_df)
```

The Distributed Advantage

`parallelism=4`: Spark trains 4 models **simultaneously** across the cluster. On a laptop, 27 models run sequentially. On a 4-worker cluster, $\sim 7\times$ faster.

ML Best Practices Checklist

- 1 **Split before fitting:** Always split data *before* fitting any transformer (prevent data leakage)
- 2 **Check class balance:** Use stratified split or class weights if imbalanced
- 3 **Use Pipeline:** Ensures consistent transforms on train and test — **critical in distributed settings**
- 4 **Multiple metrics:** Evaluate with AUC, F1, precision, recall — not just accuracy
- 5 **Cache training data:** `train_df.cache()` before `.fit()` — ML algorithms are iterative, reads data many times
- 6 **Tune hyperparameters:** Use `CrossValidator` to find optimal settings — exploits cluster parallelism
- 7 **Save model:** `model.save("hdfs:///models/...")` for reuse without retraining

Data Leakage (Reminder)

If you fit a `StringIndexer` on the **full dataset** (before splitting), the test set “leaks” information into the model. Always fit inside a Pipeline that sees only `train_df`. *Same rule as scikit-learn — more dangerous at scale because bugs are harder to spot.*

Saving & Loading Models

In *scikit-learn*: `joblib.dump(model, "model.pkl")`. In *Spark*:

```
# Save the trained pipeline model to HDFS
model.save("hdfs:///models/arrest_predictor_rf")

# Later: Load without retraining
from pyspark.ml import PipelineModel
loaded = PipelineModel.load(
    "hdfs:///models/arrest_predictor_rf")

# Use immediately on new data
new_predictions = loaded.transform(new_data)
```

Production Workflow

- 1 Train pipeline once on cluster (expensive, maybe 30 min)
- 2 Save model to HDFS — accessible to **any machine** in the cluster
- 3 Load in production app (cheap, fast, milliseconds)
- 4 Retrain periodically with new data (weekly/monthly)

🔑 Why HDFS Matters

Unlike `joblib.dump()` which saves to **one machine**, Spark saves to **HDFS** — any node in the cluster can load the model. No file copying needed.

Outline

- 1 From Traditional ML to Distributed ML
- 2 The ML Pipeline API
- 3 Feature Engineering
- 4 Model Training
- 5 Model Evaluation
- 6 Model Comparison & Hyperparameter Tuning
- 7 Summary**

Key Takeaways

- 1 **Same math, different execution:** ML algorithms don't change — the **data distribution** and **parallel training** are what's new
- 2 **Spark MLlib** = distributed ML on DataFrames (`pyspark.ml`)
- 3 **Pipeline API:** Transformer → Estimator → Model — like sklearn pipelines, but essential for distributed consistency
- 4 **Feature engineering:** StringIndexer + VectorAssembler — same concept as LabelEncoder + column selection
- 5 **Three models:** LR (baseline) → RF (non-linear) → GBT (best accuracy)
- 6 **Evaluation:** AUC, F1, confusion matrix — same metrics, computed in parallel
- 7 **CrossValidator:** GridSearchCV equivalent — parallelized across cluster



The Big Picture

HDFS → MapReduce → Hive → Spark → **MLlib**

From *storing* data to *predicting* outcomes — the complete big data pipeline.

What's Next

Wednesday — Hands-On Lab (Session 9B)

- Build the complete arrest prediction pipeline yourself
- Data prep → Feature engineering → RF → LR → GBT → Compare
- Experiment with CrossValidator and hyperparameter tuning

Week 10: Apache Kafka + Spark Structured Streaming

- Shift from **batch** to **streaming** processing
- Kafka architecture: Brokers, Topics, Producers, Consumers
- Spark Structured Streaming: real-time analytics on infinite data

The Journey So Far

Store (HDFS) → Process (MapReduce) → Query (Hive) → Analyze (Spark) → **Predict (MLlib)** → *Stream (Kafka, Week 10)*